Поиск и сортировки

линейный и бинарный поиск, асимптотическая сложность,
«долгие» сортировки

Задача о поиске двух комплементарных

Задача

В некоторой системе есть набор собственных колебаний, которые характеризуются частотой и волновым числом. Данные об этих колебаниях занесены в два массива. Требуется определить, существует ли в системе набор из двух волн, которые могут быть образованы распадом некоторой третьей, заданной волны. Будем считать, что частота и волновое число выражены в некоторых условных целых единицах в диапазоне от -2^{30} до 2^{30}.

$$k_i + k_j = k_0$$
$$\omega_i + \omega_j = \omega_0$$

Программная модель

```
1  #include <iostream>
2
3  using namespace std;
4
5  void load_data(int &omega_0, int &k_0, int omega[], int k[], unsigned size);
6
7  int main(int argc, const char * argv[]) {
8      unsigned const size = 10;
9      int omega_0, omega[size], k_0, k[size];
10
11     load_data(omega_0, k_0, omega, k,size);
12
13     return 0;
14 }
```

массивы для хранения данных о колебаниях

частота и волновое число заданной волны

функция отвечает за загрузку данных о заданной волне и колебаниях

следует правильно подобрать тип данных

Вариант исполнения загрузки данных

```
36  #include <iostream>
37
38  void load_data(int &omega_0, int &k_0, int omega[], int k[], unsigned size) {
39      cin >> omega_0 >> k_0;
40      for (unsigned idx = 0; idx != size; ++idx)
41          cin >> omega[idx] >> k[idx];
42  }
```

```
44  #include <fstream>
45
46  void load_data(int &omega_0, int &k_0, int omega[], int k[], unsigned size) {
47      ifstream fin("./in.txt");
48      fin >> omega_0 >> k_0;
49      for (unsigned idx = 0; idx != size; ++idx)
50          fin >> omega[idx] >> k[idx];
51  }
```

Поиск подходящей пары

```
14      //search
15      unsigned lha_idx_res = 0, rha_idx_res = 0;
16      bool found = false;
17      for (unsigned lha_idx = 0; lha_idx != size && !found; ++lha_idx)
18          for (unsigned rha_idx = lha_idx + 1; rha_idx != size && !found; ++lha_idx)
19              if (omega[lha_idx] + omega[rha_idx] == omega_0 && k[lha_idx] + k[rha_idx] == k_0) {
20                  lha_idx_res = lha_idx;
21                  rha_idx_res = rha_idx;
22                  found = true;
23              }
```

Поиск подходящей пары осуществляется простым перебором всех возможных пар и проверкой условий для каждой из них.

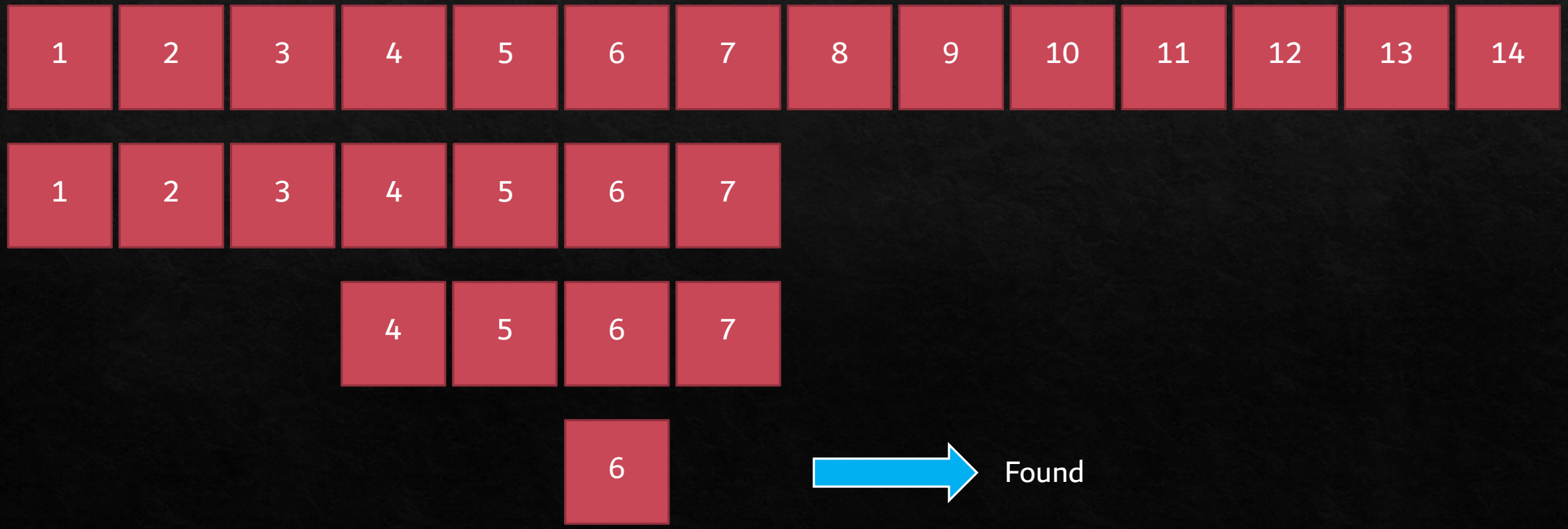
Цикл, который выбирает одну из волн как опорную.

Цикл, который последовательно перебирает все волны и проверяет для них условие. Такой поиск совпадений будем называть «линейным».

Бинарный поиск

Предыдущий поиск не подразумевал ничего о данных, которые поступают к нам в программу. Предположим, что данные к нам приходят в упорядоченном виде: колебания с меньшими частотами идут раньше колебаний с более высокими частотами. Такие данные будем называть «отсортированными».

Попробуем найти в отсортированных данных число 6. Будем проверять является ли искомое число меньше опорного – середины массива.



Реализация

```
 5  bool binary_search(int data[], unsigned size, int key) {
 6      unsigned l_idx = 0, r_idx = size;
 7
 8      while (l_idx < r_idx) {
 9          unsigned pivot_idx = (l_idx + r_idx) / 2;
10          if (key < data[pivot_idx]) {
11              r_idx = pivot_idx;
12          } else if (data[pivot_idx] < key) {
13              l_idx = pivot_idx + 1;
14          } else {
15              return true;
16          }
17      }
18
19      return false;
20  }
```

Реализация

```
39  unsigned binary_search(int data[], unsigned l_idx, unsigned r_idx, int key) {
40      auto fail_idx = r_idx;
41      while (l_idx < r_idx) {
42          unsigned pivot_idx = (l_idx + r_idx) / 2;
43          if (key < data[pivot_idx]) {
44              r_idx = pivot_idx;
45          } else if (data[pivot_idx] < key) {
46              l_idx = pivot_idx + 1;
47          } else {
48              return pivot_idx;
49          }
50      }
51
52      return fail_idx;
53  }
```

Решение с бинарным поиском

```
44      //search
45      unsigned lha_idx_res = 0, rha_idx_res = 0;
46      bool found = false;
47      for (unsigned lha_idx = 0; lha_idx != size; ++lha_idx) {
48          auto to_search = omega_0 - omega[lha_idx];
49          auto rha_idx = binary_search(omega,lha_idx+1,size,to_search);
50          if (rha_idx != size && k[lha_idx] + k[rha_idx] == k_0 ) {
51              found = true;
52              lha_idx_res = lha_idx;
53              rha_idx_res = rha_idx;
54              break;
55          }
56      }
```

Что использовать?

Когда у одной задачи есть несколько решений, то было бы интересно понять, какое из этих решений выгоднее: быстрее выполняется или использует меньше памяти.

Сложно ориентироваться на отдельные запуски программы, так как они могут зависеть от конкретного оборудования и от конкретных поданных данных.

Асимптотическая сложность алгоритма

Что использовать?

В решении этого вопроса нам поможет понятие «асимптотической сложности», которая показывает изменение времени или памяти, требуемого на исполнение алгоритма, при увеличении размера входящих данных.

Назовём функцию f ограниченной сверху функцией g с точностью до множителя если $\exists C > 0, N : \forall n > N \; f(n) < Cg(n)$ Будем записывать это как $f \in O(g)$, где $O(g)$ - множество всех функций для которых выполняется определение выше.

Если мы выразим время исполнения алгоритма или используемую им память в некоторых условных единицах, не зависящих от оборудования, то функция g может показать нам насколько быстро растут требования к оборудованию с ростом данных, за исключением, возможно, небольшого их количества, для которого можно протестировать алгоритм отдельно.

Асимптотическая сложность линейного поиска

Для оценки скорости будем подсчитывать операции над данными. Подсчитывать мы будем арифметические операции (A) и операции чтения/записи (L/S) в память. Используя этот подход, оценим количество операций для линейного поиска.

```
 5  bool linear_search(int data[], unsigned size, int key) {
 6      for (unsigned idx = 0; idx != size; ++idx)
 7          if (key == data[idx])
 8              return true;
 9      return false;
10  }
```

$$OPERATIONS = COUNT * 2$$

Цикл выполняется некоторое количество раз COUNT. Это зависит от того, где находится искомый элемент: если в самом начале, то 1 раз, а если вообще не содержится, то ровно N раз, где N – количество элементов в массиве.

На каждой итерации цикла мы производим одну операцию загрузки (1L) и одну операцию сравнения (1A)

Наилучший, наихудший, средний

Когда точное количество операций зависит от самих данных, принято выделять три случая: наилучший – минимальное количество операций, наихудший – максимальное количество операций, средний – оценка количества операций для произвольных данных.

```
 5  bool linear_search(int data[], unsigned size, int key) {
 6      for (unsigned idx = 0; idx != size; ++idx)
 7          if (key == data[idx])
 8              return true;
 9      return false;
10  }
```

$COUNT \leftarrow 1$ наилучший

$COUNT \leftarrow N$ наихудший

$$\frac{1 + 2 + \cdots + (N-1) + N}{N} = \frac{N(N+1)}{2N} = \frac{N+1}{2}$$ средний

$$OPERATIONS = COUNT * 2 = 2 * \frac{N+1}{2} = N + 1$$ средний

$f(N) = (N+1) \rightarrow f(N) \in O(N)$

$f(N) \in O(1)$ Постоянная, константная сложность. Не зависит от количества входящих данных.

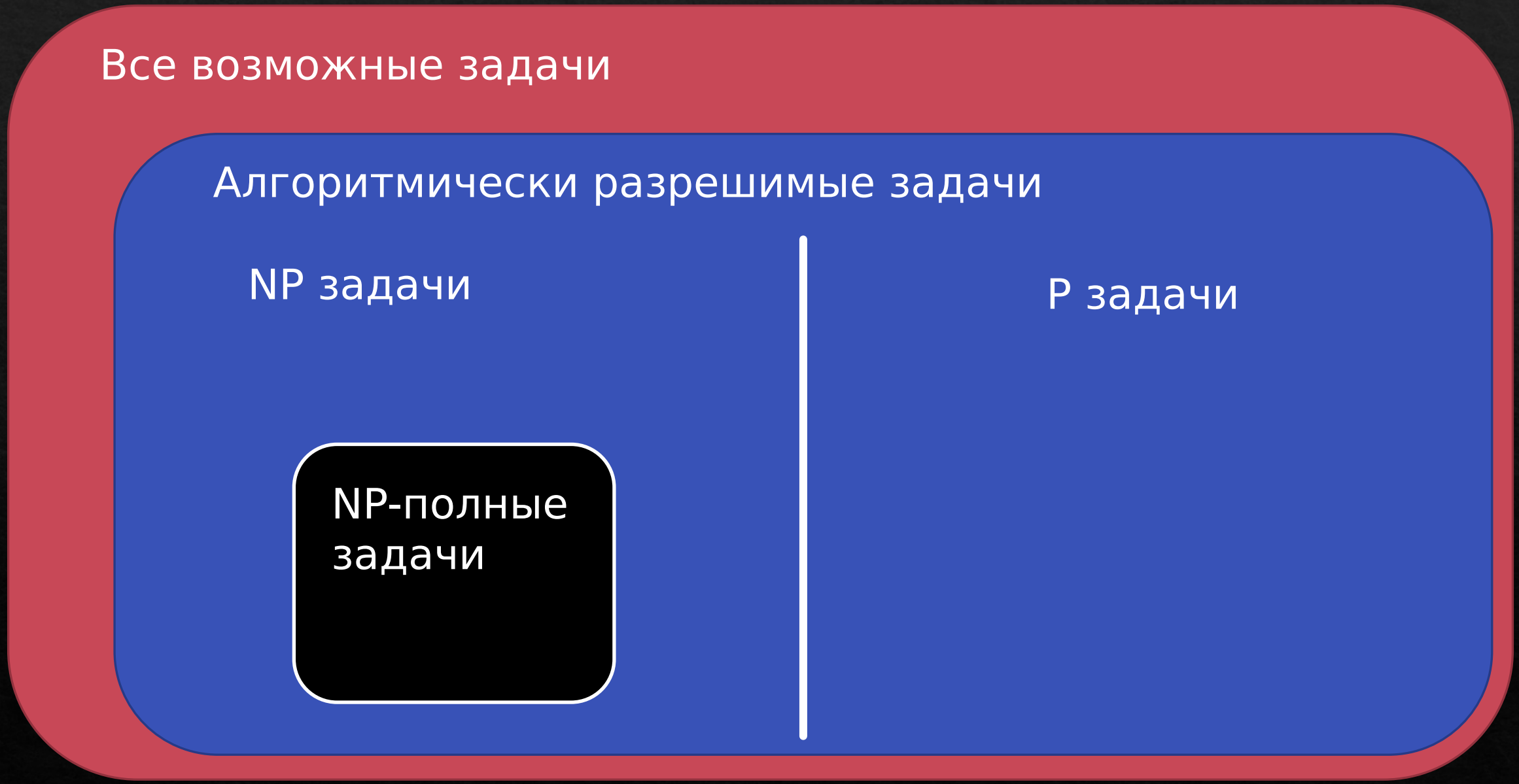
$f(N) \in O(N)$ Линейная сложность. Количество операций растёт прямо пропорционально количеству входящих данных.

$f(N) \in O(N^2)$ Квадратичная сложность. Количество операций растёт быстрее, чем количество входящих данных.

$f(N) \in O(\log N)$ Логарифмическая сложность. Основание логарифма не важно, так как класс *O(*log *N)* рассматривает ограничение с точностью до константы.

$f(N) \in O(2^N)$ Экспоненциальная, не полиномиальная (NP) сложность.

Классы сложности



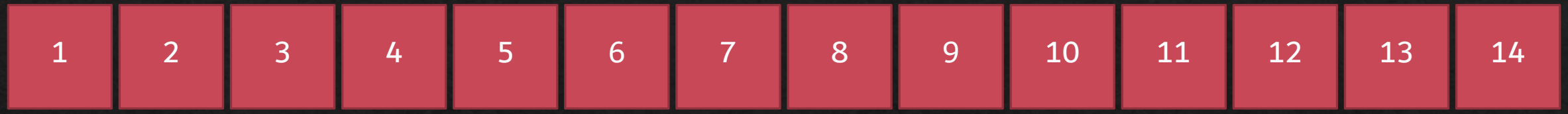
Асимптотическая сложность бинарного поиска

```
39  unsigned binary_search(int data[], unsigned l_idx, unsigned r_idx, int key) {
40      auto fail_idx = r_idx;
41      while (l_idx < r_idx) {
42          unsigned pivot_idx = (l_idx + r_idx) / 2;
43          if (key < data[pivot_idx]) {
44              r_idx = pivot_idx;
45          } else if (data[pivot_idx] < key) {
46              l_idx = pivot_idx + 1;
47          } else {
48              return pivot_idx;
49          }
50      }
51
52      return fail_idx;
53  }
```

Цикл выполняется некоторое количество раз COUNT. Это зависит от того, где находится искомый элемент.

На каждой итерации цикла мы производим не более пяти арифметических операций (5A) и не более трёх операций чтения/записи (3L/S). Полное количество операций не зависит от числа элементов и принадлежит *O(1)*.

Наихудший, наилучший, средний



$COUNT \leftarrow 1$

В наилучшем случае мы сразу попадаем в искомый элемент

$COUNT \leftarrow \log N$

В наихудшем случае цикл продолжается с уменьшением количества элементов вдвое.

Средний случай $\in O(\log N)$

Свойства классов ограничений

$f \in O(h), g \in O(h) \rightarrow (f + g) \in O(h)$

$f \in O(h), g \in O(l) \rightarrow (f * g) \in O(h * l)$

$f \in O(h), g \in O(l), h \in O(l) \rightarrow (f + g) \in O(l)$

Асимптотическая сложность бинарного поиска

```
39  unsigned binary_search(int data[], unsigned l_idx, unsigned r_idx, int key) {
40      auto fail_idx = r_idx;
41      while (l_idx < r_idx) {
42          unsigned pivot_idx = (l_idx + r_idx) / 2;
43          if (key < data[pivot_idx]) {
44              r_idx = pivot_idx;
45          } else if (data[pivot_idx] < key) {
46              l_idx = pivot_idx + 1;
47          } else {
48              return pivot_idx;
49          }
50      }
51
52      return fail_idx;
53  }
```

O(log N) x O(1) = O(log N)

```
14      //search
15      unsigned lha_idx_res = 0, rha_idx_res = 0;
16      bool found = false;
17      for (unsigned lha_idx = 0; lha_idx != size && !found; ++lha_idx)
18          for (unsigned rha_idx = lha_idx + 1; rha_idx != size && !found; ++lha_idx)
19              if (omega[lha_idx] + omega[rha_idx] == omega_0 && k[lha_idx] + k[rha_idx] == k_0) {
20                  lha_idx_res = lha_idx;
21                  rha_idx_res = rha_idx;
22                  found = true;
23              }
```

```
44      //search
45      unsigned lha_idx_res = 0, rha_idx_res = 0;
46      bool found = false;
47      for (unsigned lha_idx = 0; lha_idx != size; ++lha_idx) {
48          auto to_search = omega_0 - omega[lha_idx];
49          auto rha_idx = binary_search(omega,lha_idx+1,size,to_search);
50          if (rha_idx != size && k[lha_idx] + k[rha_idx] == k_0 ) {
51              found = true;
52              lha_idx_res = lha_idx;
53              rha_idx_res = rha_idx;
54              break;
55          }
56      }
```

Сортировка

Сортировка пузырьком

```
 6  void ascending_bubble_sort(int data[], unsigned l_idx, unsigned r_idx) {
 7      bool sorted;
 8      do  {
 9          sorted = true;
10          for (unsigned curr = l_idx, next = l_idx + 1; next != r_idx; ++curr, ++next)
11              if (data[curr] > data[next]) {
12                  swap(data[curr], data[next]);
13                  sorted = false;
14              }
15      } while (!sorted);
16  }
```

Какова её асимптотическая сложность в предположении, что функция swap имеет сложность *O(1)*?

Добавим сортировку в основной алгоритм

```
 82      unsigned const size = 10;
 83      int omega_0, omega[size], k_0, k[size];
 84      load_data(omega_0, k_0, omega, k,size);
 85
 86      ascending_bubble_sort_alt(omega,k,0,size);
 87
 88      //search
 89      unsigned lha_idx_res = 0, rha_idx_res = 0;
 90      bool found = false;
 91      for (unsigned lha_idx = 0; lha_idx != size; ++lha_idx) {
 92          auto to_search = omega_0 - omega[lha_idx];
 93          auto rha_idx = binary_search(omega,lha_idx+1,size,to_search);
 94          if (rha_idx != size) {
 95              found = true;
 96              lha_idx_res = lha_idx;
 97              rha_idx_res = rha_idx;
 98              break;
 99          }
100      }
```

Какова асимптотическая сложность полного алгоритма?

Понятие устойчивости сортировки

Сортировка называется «устойчивой», если порядок равных элементов в отсортированной последовательности такой же, как и в изначальной последовательности.

